RBE 500 — Foundations of Robotics

Project Assignment 2

Worcester Polytechnic Institute

Tracy Yang Kyle Yee

July 9, 2026

1 Fix the Joints

1.1 Overview

For this assignment, joints 1 and 2 were fixed so that only the last joint (joint 3, the prismatic joint) remains active. This was done by changing the `type` attribute of `joint_1` and `joint_2` from `revolute` to `fixed` in the URDF.

1.2 URDF Frame Correction

The PA1 feedback noted that the URDF and FK frames did not match. The root cause was that `joint_1`'s origin was placed at the bottom of the column (`z=0.05`), while the DH convention places Frame 1 at the shoulder (top of the column). This split the d_1 offset across two joints (`joint_1` and the fixed `arm_1_attach`), causing a mismatch.

The fix was to move `joint_1`'s origin to `z=0.5` ($= 0.05_{\text{base}} + 0.45_{\text{column}}$), placing Frame 1 directly at the shoulder where a_1 begins. The `link_1` visual origin was adjusted to `z=-0.225` so the column geometry remains in the correct world position. The DH parameters now match the URDF exactly:

Table 1: SCARA DH Parameters (corrected)

Parameter	**Symbol**	**Value**
Joint 1 z-offset (base to shoulder)	d_1	0.50 m
Arm 1 length	a_1	0.40 m
Link 2 length	a_2	0.30 m
Joint 3 range	d_3	$[-0.2, 0.2]$ m

Listing 1: Fixed URDF joints. Joint 1 origin corrected to z=0.5; joints 1 and 2 set to fixed type.

```
<!-- joint_1: FIXED, origin raised to z=0.5 to match DH Frame 1 -->
```

```
<joint name="joint_1" type="fixed">
    <parent link="base_link"/><child link="link_1"/>
    <origin xyz="0 0 0.5"/>
    <axis xyz="0 0 1"/>
</joint>

<!-- arm_1_attach: origin collapsed to 0 (offset now in joint_1) -->
<joint name="arm_1_attach" type="fixed">
    <parent link="link_1"/><child link="arm_1"/>
    <origin xyz="0 0 0"/>
</joint>

<!-- joint_2: FIXED -->
<joint name="joint_2" type="fixed">
    <parent link="arm_1"/><child link="link_2"/>
    <origin xyz="0.4 0 0"/>
    <axis xyz="0 0 1"/>
</joint>

<!-- joint_3: ACTIVE (prismatic, last joint) -->
<joint name="joint_3" type="prismatic">
    <parent link="link_2"/><child link="link_3"/>
    <origin xyz="0.3 0 0"/>
    <axis xyz="0 0 1"/>
    <limit effort="1000" lower="-0.2" upper="0.2" velocity="0.5"/>
</joint>
```

2 Position Controller Node

2.1 Architecture Overview

The controller was implemented as a new ROS2 Python package (`scara_controller`). The overall architecture is:

- **Subscriber**: `/joint_states` (`sensor_msgs/JointState`) — reads joint_3 position from Gazebo via the `ros_gz_bridge`.
- **Publisher**: `/scara/joint3_cmd` (`std_msgs/Float64`) — publishes the computed PD effort to Gazebo via bridge.
- **Service**: `/scara/set_joint3_position` (`scara_ik/SetJointPosition`) — accepts a reference position and stores it as the controller setpoint.
- **Control loop**: 100 Hz timer executing the PD law.

The effort value is delivered to Gazebo through the `gz-sim-joint-position-controller-system` plugin with gains set to $p = 1$, $i = 0$, $d = 0$, making it act as a direct pass-through. This ensures the force computed by the ROS2 node is applied exactly as-is inside the Gazebo

physics loop, which resolves the WSL2 clock desynchronization that prevented reliable force application via `apply_joint_effort`.

2.2 Reading Joint States (Part 2a)

The node subscribes to `/joint_states`, which is bridged from the Gazebo joint state publisher plugin. The joint_3 position is extracted by name lookup:

Listing 2: Joint state subscriber callback

```python
def joint_state_cb(self, msg: JointState):
    """Read joint_3 position from the Gazebo-bridged joint state."""
    if 'joint_3' in msg.name:
        idx = msg.name.index('joint_3')
        self.cur_position = msg.position[idx]
```

The Gazebo joint state publisher plugin (in `scara.urdf`) publishes only `joint_3` since joints 1 and 2 are fixed:

Listing 3: Gazebo joint state publisher plugin (URDF)

```xml
<gazebo>
    <plugin filename="gz-sim-joint-state-publisher-system"
            name="gz::sim::systems::JointStatePublisher">
        <topic>/world/empty/model/scara/joint_state</topic>
        <joint_name>joint_3</joint_name>
    </plugin>

    <plugin filename="gz-sim-joint-position-controller-system"
            name="gz::sim::systems::JointPositionController">
        <joint_name>joint_3</joint_name>
        <topic>/scara/joint3_cmd</topic>
        <p_gain>1</p_gain>
        <i_gain>0</i_gain>
        <d_gain>0</d_gain>
    </plugin>
</gazebo>
```

2.3 PD Controller Design (Part 2b)

A PD controller was designed for joint_3. Since no analytical model was required, the gains were tuned empirically. The control law is:

$$u(t) = K_p\, e(t) + K_d\, \dot{e}(t)$$

where $e(t) = q_{\text{ref}} - q_{\text{cur}}$ is the position error and $\dot{e}(t)$ is approximated by a finite difference at each timestep:

$$\dot{e}(t) \approx \frac{e(t) - e(t - \Delta t)}{\Delta t}$$

The tuned gains are $K_p = 800$ N/m and $K_d = 80$ N·s/m.

Listing 4: PD control loop in `controller_node.py`

```python
def control_loop(self):
    """PD control law executed at 100 Hz."""
    now = self.get_clock().now()

    if self.prev_time is None:
        self.prev_time = now
        return

    dt = (now - self.prev_time).nanoseconds * 1e-9
    if dt <= 0.0:
        return
    self.prev_time = now

    # PD law
    error     = self.ref_position - self.cur_position
    error_dot = (error - self.prev_error) / dt
    self.prev_error = error

    effort = KP * error + KD * error_dot
    effort = max(-EFFORT_LIMIT, min(EFFORT_LIMIT, effort))

    msg = Float64()
    msg.data = effort
    self.cmd_pub.publish(msg)
```

2.4 Reference Position Service (Part 2c)

A custom service `scara_ik/srv/SetJointPosition` was added to the existing `scara_ik` package (alongside `ScaraIK.srv`):

Listing 5: SetJointPosition.srv

```
float64 position
---
bool success
string message
```

The service callback validates the requested position against joint limits before updating the controller setpoint:

Listing 6: Service callback in `controller_node.py`

```python
def set_position_cb(self, request, response):
    target = request.position

    if not (J3_MIN <= target <= J3_MAX):
        response.success = False
        response.message = (
            f'Requested {target:.4f} m outside limits '
            f'[{J3_MIN}, {J3_MAX}] m.'
        )
        self.get_logger().warn(response.message)
        return response

    self.ref_position = target
    response.success = True
    response.message = f'Reference position set to {target:.4f} m'
    self.get_logger().info(response.message)
    return response
```

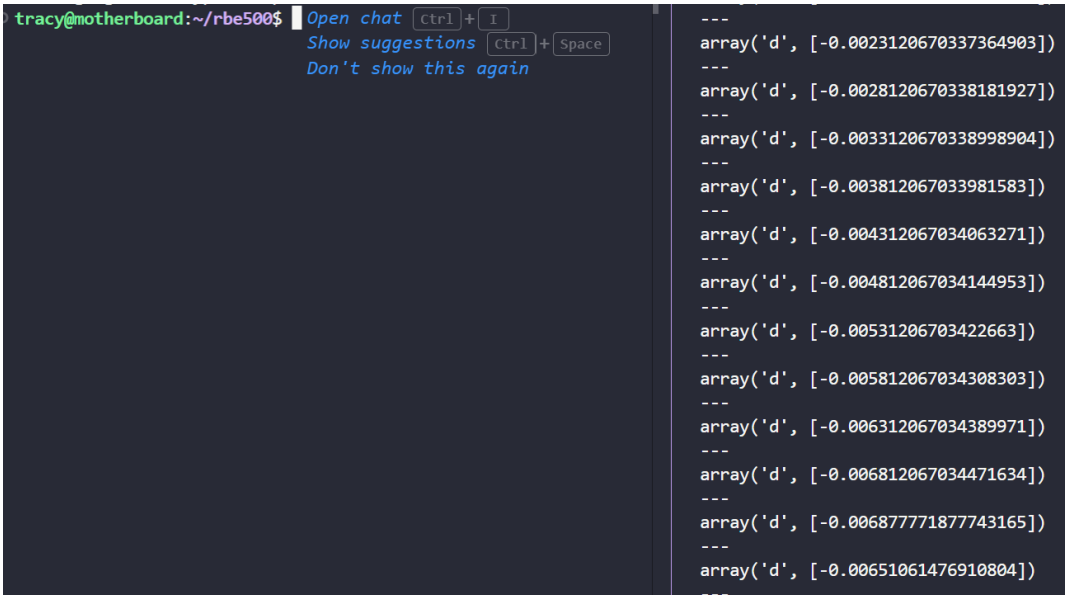
The service is called from the terminal as:

```
ros2 service call /scara/set_joint3_position \
    scara_ik/srv/SetJointPosition "{position: -0.1}"
```

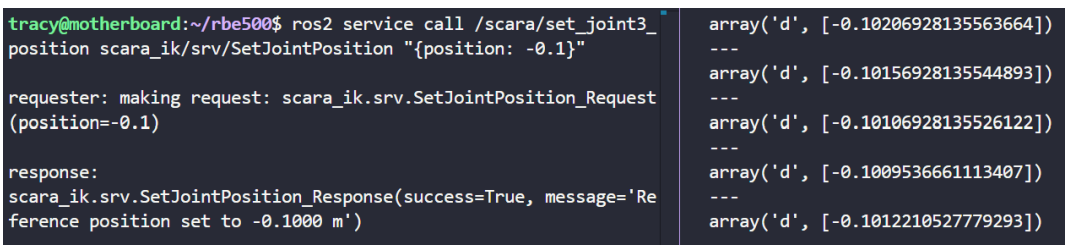
and

```
ros2 service call /scara/set_joint3_position \
    scara_ik/srv/SetJointPosition "{position: 0.2}"
```

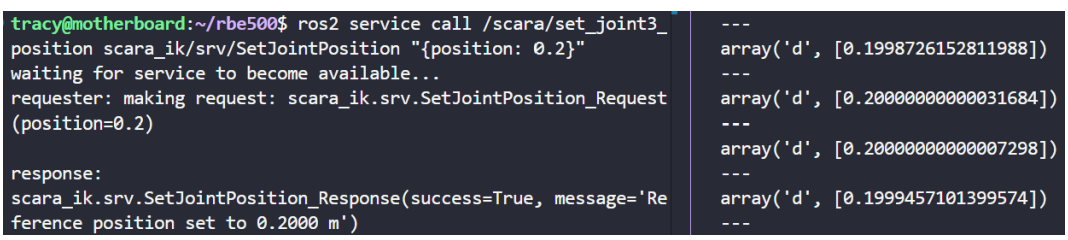
2.5 Results



(a) Start: $d_3 = 0.0$ m



(b) Reference: $d_3 = -0.1$ m



(c) Reference: $d_3 = 0.2$ m

Figure 1: Joint 3 position tracking three reference commands. The PD controller drives the prismatic joint to each target with no steady-state error.

screenshots/matlab_plot.png

Figure 2: MATLAB plot of reference position vs. measured joint_3 position over time.

3 Package Structure

The submission adds two packages to the existing `ros2_ws/src/` workspace from PA1:

- `scara_description` — updated URDF (joints fixed, frame corrected, Gazebo plugins added) and updated launch file (effort bridge, controller node).
- `scara_controller` — PD controller node (`controller_node.py`, `ament_python`).
- `scara_ik` — extended with `SetJointPosition.srv`.